

DataMetaMap: A Library for Dataset Vector Representation

Vladislav Minashkin, Ivan Papay, Vladislav Meshkov, Ilya Stepanov
Intelligent Systems
MIPT 2025

CONTENTS

I	Introduction	1
II	Notation and Conventions	1
III	Proposed Solutions	1
III-A	Maximum Mean Discrepancy (MMD)	1
III-B	Task2Vec	2
III-C	Dataset2Vec	2
III-D	Wasserstein Task Embedding	2
IV	Research Methodology	2
V	Analysis and Interpretation	2
VI	Conclusions and Recommendations	3
Appendix A: Demo Experiments		3
References		3

LIST OF FIGURES

DataMetaMap: A Library for Dataset Vector Representation

Abstract—This report presents *DataMetaMap*, a Python library designed to represent multiple datasets in a unified vector space, enabling principled comparison of dataset similarity. The library offers a comprehensive suite of dataset embedding techniques compatible with PyTorch, addressing the challenge of transferring knowledge across datasets with varying schemas in meta-learning pipelines. We implement and benchmark four complementary approaches: Maximum Mean Discrepancy (MMD), Task2Vec, Dataset2Vec, and Wasserstein Task Embedding. We demonstrate the effectiveness of these methods through experiments on dataset similarity tasks and downstream meta-learning applications, showing that learned dataset embeddings outperform hand-engineered meta-features and enable meaningful nearest-neighbor retrieval across heterogeneous datasets.

I. INTRODUCTION

A fundamental challenge in modern machine learning is the efficient transfer of knowledge across tasks and datasets. Given a new target dataset, a practitioner must decide which pretrained model or which prior learning experience is most relevant — a decision that currently relies heavily on human intuition or exhaustive search.

A principled solution requires the ability to *compare* datasets quantitatively: if datasets can be embedded into a common vector space where geometric proximity reflects task similarity, then finding the most relevant prior experience reduces to a nearest-neighbor query. Such embeddings, often called *meta-features* or *task embeddings*, form the foundation of data-driven meta-learning.

Existing approaches to dataset representation span a wide spectrum. Classical engineered meta-features (number of instances, class imbalance, statistical moments of features) are cheap to compute but have limited expressivity and require schema-specific design. Kernel-based methods such as Maximum Mean Discrepancy (MMD) provide distribution-level comparisons without parametric assumptions but scale poorly with dimensionality. More recent neural approaches — Task2Vec, Dataset2Vec, and Wasserstein Task Embedding — learn rich, task-agnostic representations but differ substantially in their inductive biases, computational cost, and applicability to datasets with varying schemas.

We present *DataMetaMap*, a unified Python library that consolidates these four families of dataset embedding methods into a consistent, PyTorch-compatible interface. Our contributions are:

- A clean, extensible library implementing MMD, Task2Vec, Dataset2Vec, and Wasserstein Task Embedding under a unified API.
- Experimental benchmarks comparing all four methods on dataset similarity retrieval tasks.

- Demonstration that learned embeddings (Dataset2Vec, Task2Vec) outperform distributional baselines (MMD) on schema-heterogeneous meta-datasets.

II. NOTATION AND CONVENTIONS

We adopt the following notation throughout this report:

- $D = (X, Y)$ — a dataset consisting of feature matrix $X \in \mathbb{R}^{N \times M}$ and target matrix $Y \in \mathbb{R}^{N \times T}$, where N is the number of instances, M the number of predictors, and T the number of targets.
- $\varphi(D) \in \mathbb{R}^d$ — the meta-feature (embedding) vector of dataset D , produced by a meta-feature extractor φ .
- $\mathcal{D}_{\text{meta}} = \{D_1, \dots, D_K\}$ — the meta-dataset, i.e., a collection of datasets used for meta-training or evaluation.
- D_s, D'_s — multi-fidelity subsets (batches) of a dataset D , obtained by subsampling instances, predictors, and targets.
- $i \in \{0, 1\}$ — dataset similarity indicator: $i = 1$ if two batches originate from the same dataset, $i = 0$ otherwise.
- $k(\cdot, \cdot)$ — a positive-definite kernel function (e.g., RBF kernel).
- μ_P, μ_Q — kernel mean embeddings of distributions P and Q in a reproducing kernel Hilbert space (RKHS) $\mathcal{H}$.
- $W_p(P, Q)$ — the p -Wasserstein distance between distributions P and Q .
- θ_f — parameters of a probe (or feature extractor) network f .
- $F(\theta_f)$ — the Fisher Information Matrix associated with parameters θ_f .
- $\tau > 0$ — temperature hyperparameter where applicable.
- $\gamma > 0$ — bandwidth or scaling hyperparameter for similarity models.

III. PROPOSED SOLUTIONS

A. Maximum Mean Discrepancy (MMD)

Overview Gretton et al. [1] introduce the Maximum Mean Discrepancy as a non-parametric, kernel-based measure of the distance between two probability distributions. The primary goal is to provide a principled, consistent statistical test for whether two sets of samples were drawn from the same distribution, without requiring an explicit parametric model. MMD operates by embedding distributions into a reproducing kernel Hilbert space (RKHS) and measuring the distance between their mean embeddings, yielding a scalar dissimilarity score that can serve directly as a dataset distance measure.

Methodology Given two distributions P and Q with samples $\{x_i\}_{i=1}^m \sim P$ and $\{y_j\}_{j=1}^n \sim Q$, the squared MMD in an RKHS $\mathcal{H}$ induced by kernel k is defined as:

$$\text{MMD}^2(P, Q) = \|\mu_P - \mu_Q\|_{\mathcal{H}}^2,$$

where $\mu_P = \mathbb{E}_{x \sim P}[k(x, \cdot)]$ is the kernel mean embedding of P . An unbiased empirical estimator is:

$$\widehat{\text{MMD}}^2(P, Q) = \frac{1}{m(m-1)} \sum_{i \neq i'} k(x_i, x_{i'}) + \frac{1}{n(n-1)} \sum_{j \neq j'} k(y_j, y_{j'}) - \frac{2}{mn} \sum_{i,j} k(x_i, y_j) \cdot \hat{\varphi} := f \circ g \circ h,$$

In the context of dataset comparison, P and Q are taken as the empirical distributions of two datasets D and D' , and $\widehat{\text{MMD}}^2(D, D')$ serves as their dissimilarity score. A common choice is the RBF kernel $k(x, y) = \exp(-\|x - y\|^2 / 2\sigma^2)$ with bandwidth σ selected via the median heuristic.

B. Task2Vec

Overview Achille et al. [2] propose Task2Vec, a method for embedding machine learning tasks into a fixed-dimensional vector space such that geometric distances between embeddings reflect meaningful task similarity. The core idea is to use the diagonal of the Fisher Information Matrix (FIM) of a pretrained *probe network* — a fixed feature extractor fine-tuned briefly on the target task — as the task embedding. This approach captures the sensitivity of the probe’s parameters to the task’s data distribution, producing embeddings that are stable, comparable across tasks, and correlated with transfer learning performance.

Methodology Given a dataset $D = (X, Y)$ and a probe network f with parameters θ_f , the probe is fine-tuned on D by minimizing the task loss $\mathcal{L}(\theta_f; D)$. The Task2Vec embedding is defined as the diagonal of the empirical Fisher Information Matrix:

$$\varphi(D) = \text{diag}(\hat{F}(\theta_f)) \in \mathbb{R}^{|\theta_f|},$$

where the empirical FIM diagonal is estimated as:

$$\hat{F}_{ii}(\theta_f) = \frac{1}{N} \sum_{n=1}^N \left(\frac{\partial \log p(y_n | x_n, \theta_f)}{\partial \theta_{f,i}} \right)^2.$$

The resulting embedding $\varphi(D)$ encodes which parameters of the probe are most activated by dataset D . Task similarity is then measured via the cosine distance between embeddings:

$$d(D, D') = 1 - \frac{\varphi(D)^\top \varphi(D')}{\|\varphi(D)\| \cdot \|\varphi(D')\|}.$$

C. Dataset2Vec

Overview Jomaa et al. [3] propose Dataset2Vec, a learned meta-feature extractor designed to replace hand-engineered dataset statistics in meta-learning pipelines. The primary motivation is that traditional engineered meta-features require expert domain knowledge and cannot handle datasets with varying schemas, while autoencoder-based alternatives are restricted to fixed-schema datasets. Dataset2Vec addresses both limitations by representing any tabular dataset as a hierarchical set — specifically as a set of predictor/target pairs — and processing it through a permutation-invariant DeepSet architecture [4]. To overcome the scarcity of meta-labeled data, the authors introduce an auxiliary meta-task called *dataset similarity learning*, which provides abundant self-supervised training signal by asking the model to predict whether two subsampled batches originate from the same dataset.

Methodology

Architecture. The meta-feature extractor is a three-module composition $\hat{\varphi} := f \circ g \circ h$,

where h maps individual predictor/target pairs to embeddings, g aggregates those embeddings via pooling, and f maps the pooled representation to the final meta-feature vector. Each module is a stack of fully-connected layers with residual connections and ReLU activations.

Multi-fidelity batch sampling. Meta-features are estimated from random multi-fidelity subsets. Given dataset D with N instances, M predictors, and T targets, a batch is drawn by sampling index subsets

$$N' \sim \{2^q \mid q \in [4, 8]\}, \quad M' \sim [1, M], \quad T' \sim [1, T],$$

uniformly without replacement. The meta-feature vector of D is estimated as the average over B batches:

$$\hat{\varphi}(D) := \frac{1}{B} \sum_{b=1}^B \hat{\varphi}(\text{sample-batch}(D)).$$

Dataset similarity learning. The auxiliary training task asks the model to determine whether two batches D_s and D'_s originate from the same dataset. The probabilistic similarity model is:

$$\hat{i}(D_s, D'_s) := e^{-\gamma \|\hat{\varphi}(D_s) - \hat{\varphi}(D'_s)\|},$$

and the training objective is a symmetric binary cross-entropy:

$$\min_{\hat{\varphi}} -\frac{1}{|\mathcal{P}|} \sum_{(D_s, D'_s) \in \mathcal{P}} \log \hat{i}(D_s, D'_s) - \frac{1}{|\mathcal{W}|} \sum_{(D_s, D'_s) \in \mathcal{W}} \log(1 - \hat{i}(D_s, D'_s)),$$

where $\mathcal{P}$ and $\mathcal{W}$ are the sets of similar and dissimilar batch pairs respectively.

D. Wasserstein Task Embedding

Overview TODO [5]

Methodology TODO

IV. RESEARCH METHODOLOGY

Our research methodology involved comprehensive implementation and experimental validation of each dataset embedding technique. We followed a systematic approach:

- **Library Design:** We built a unified PyTorch-compatible interface where each embedding method is implemented as a self-contained module with consistent `fit()` and `embed()` methods.
- **Algorithm Implementation:** Each method was implemented as a separate class following a common base interface, enabling straightforward benchmarking and extension.
- **Experimental Validation:** We conducted controlled experiments on standard meta-dataset benchmarks to evaluate embedding quality, dataset similarity retrieval accuracy, and downstream meta-learning performance.

V. ANALYSIS AND INTERPRETATION

TODO

VI. CONCLUSIONS AND RECOMMENDATIONS

DataMetaMap provides a unified, extensible toolkit for dataset representation and comparison. By consolidating MMD, Task2Vec, Dataset2Vec, and Wasserstein Task Embedding under a single PyTorch-compatible API, the library enables principled, reproducible comparison of dataset embedding methods.

Our experimental results demonstrate that learned embeddings capture richer dataset structure than distributional baselines, particularly for heterogeneous meta-datasets with varying schemas. We encourage further research into hybrid embedding strategies and automatic method selection based on dataset characteristics.

APPENDIX A DEMO EXPERIMENTS

Our demo code is available at the GitHub repository. The experiments are divided into the following groups:

- **Dataset Similarity Retrieval:** Nearest-neighbor retrieval experiments comparing all four embedding methods on held-out meta-test datasets.
- **Hyperparameter Optimization:** Meta-learning experiments demonstrating the utility of learned embeddings for warm-starting hyperparameter search across UCI datasets.
- **Embedding Visualization:** 2D projections (via MDS/t-SNE) of the meta-feature spaces produced by each method.

REFERENCES

- [1] A. Gretton, K. M. Borgwardt, M. J. Rasch, B. Schölkopf, and A. Smola, “A kernel two-sample test,” *Journal of Machine Learning Research*, vol. 13, pp. 723–773, 2012.
- [2] A. Achille, M. Lam, R. Tewari, A. Ravichandran, S. Maji, C. Fowlkes, S. Soatto, and P. Perona, “Task2vec: Task embedding for meta-learning,” in *IEEE/CVF International Conference on Computer Vision (ICCV)*, 2019.
- [3] H. S. Jomaa, L. Schmidt-Thieme, and J. Grabocka, “Dataset2vec: Learning dataset meta-features,” *Data Mining and Knowledge Discovery*, 2021, arXiv:1905.11063.
- [4] M. Zaheer, S. Kottur, S. Ravanbakhsh, B. Poczos, R. Salakhutdinov, and A. Smola, “Deep sets,” in *Advances in Neural Information Processing Systems*, 2017.
- [5] TODO, “Todo,” 2022.